

Modify the language to allow functions to take one or more arguments. Start by modifying the abstract syntax in `Absyn.fs` to permit a list of parameter names in `Letfun` and a list of argument expressions in `Call`.

Then modify the `eval` interpreter in file `Fun.fs` to work for the new abstract syntax. You must modify the closure representation to accommodate a list of parameters. Also, modify the `Letfun` and `Call` clauses of the interpreter. You will need a way to zip together a list of variable names and a list of variable values, to get an environment in the form of an association list; so function `List.zip` might be useful.

Exercise 4.4 In continuation of Exercise 4.3, modify the parser specification to accept a language where functions may take any (non-zero) number of arguments. The resulting parser should permit function declarations such as these:

```
let pow x n = if n=0 then 1 else x * pow x (n-1) in pow 3 8 end

let max2 a b = if a<b then b else a
in let max3 a b c = max2 a (max2 b c)
  in max3 25 6 62 end
end
```

You may want to define non-empty parameter lists and argument lists in analogy with the `Names1` nonterminal from `Usql/UsqlPar.fsy`, except that the parameters should not be separated by commas. Note that multi-argument applications such as `f a b` are already permitted by the existing grammar, but they would produce abstract syntax of the form `Call(Call(Var "f", Var "a"), Var "b")` which the `Fun.eval` function does not understand. You need to modify the `AppExpr` nonterminal and its semantic action to produce `Call(Var "f", [Var "a"; Var "b"])` instead.

Exercise 4.5 Extend the (untyped) functional language with infix operator “&&” meaning sequential logical “and” and infix operator “|” meaning sequential logical “or”, as in C, C++, Java, C#, or F#. Note that `e1 && e2` can be encoded as `if e1 then e2 else false` and that `e1 || e2` can be encoded as `if e1 then true else e2`. Hence you need only change the lexer and parser specifications, and make the new rules in the parser specification generate the appropriate abstract syntax. You need not change `Absyn.fs` or `Fun.fs`.

Exercise 4.6 Extend the abstract syntax, the concrete syntax, and the interpreter for the untyped functional language to handle tuple constructors `(...)` and component selectors `#i` (where the first component is `#1`):

```
type expr =
  | ...
  | Tup of expr list
  | Sel of int * expr
  | ...
```

- Function `mapTree : ('a -> 'b) -> ('a tree -> 'b tree)` which applies the function to each node of the tree, and returns a tree of the same shape with the new node values.

The preorder, inorder, and postorder traversals are defined in Exercise A.3.

Exercise 5.6 This exercise is about using higher-order functions for production of HTML code. This is handy when generating static webpages from database information and when writing Web scripts in a functional style:

(A) Write an F# function

```
htmlrow : int * (int -> string) -> string
```

that builds one row of a numeric HTML table (with right-aligned table data). For example,

```
htmlrow (2, fun j -> string(j * 8))
```

should produce this string:

```
"<td align=right>0</td><td align=right>8</td>"
```

Write an F# function

```
htmltable : int * (int -> string) -> string
```

that builds an HTML table. For example,

```
htmltable (3, fun i -> "<td>" + string(i) + "</td>" +  
                      "<td>" + string(i*8) + "</td>");
```

should produce an F# string that will print like this, including line breaks:

```
<table>  
<tr><td>0</td><td>0</td></tr>  
<tr><td>1</td><td>8</td></tr>  
<tr><td>2</td><td>16</td></tr>  
</table>
```

Newlines are represented by `\n` characters as in C, Java and C#. Similarly,

```
htmltable (10,  
          fun i -> htmlrow(10,  
                          fun j -> string((i+1)*(j+1))))
```

should produce a 10-by-10 multiplication table in HTML.

(B) Implement methods similar to `htmlrow` and `htmltable` in Java. (This is cumbersome, but instructive).

Exercise 5.7 Extend the monomorphic type checker to deal with lists. Use the following extra kinds of types:

```

type typ =
  | ...
  | TypL of typ          (* list, element type is typ *)
  | ...

```

Exercise 5.8 Study a lazy functional language such as Haskell [4].

Exercise 5.9 Study the implementation of a lazy language by reading Peyton Jones and Lester’s book *Implementing functional languages* [6], or Sestoft’s paper *Deriving a lazy abstract machine* [8]. Implement an interpreter for a small lazy functional language in F#. Inspiration for lexer and parser specifications, as well as abstract syntax, may be found in directory `mosml/examples/lex yacc/` in the Moscow ML distribution [5].

References

1. Barendregt, H.: The lambda calculus: its syntax and semantics. In: *Studies in Logic and the Foundations of Mathematics*, vol. 103, revised edn. North-Holland (1984)
2. Church, A.: An unsolvable problem of elementary number theory. *Am. J. Math.* **58**(2), 345–363 (1936)
3. Dean, J., Ghemawat, S.: Mapreduce: simplified data processing on large clusters. In: *OSDI 2004* (2004)
4. Haskell programming language. <http://www.haskell.org/>
5. Moscow ML: <http://mosml.org/>
6. Peyton Jones, S., Lester, D.: *Implementing Functional Languages*. Prentice-Hall (1992)
7. Sestoft, P.: Lambda calculus reduction workbench. Web page (1996). <http://www.itu.dk/people/sestoft/lamreduce/>
8. Sestoft, P.: Deriving a lazy abstract machine. *J. Funct. Program.* **7**(3), 231–264 (1997)
9. Sestoft, P.: *Java Precisely*, 3rd edn. The MIT Press, Cambridge (2016)

6.7 History and Literature

ML-style parametric polymorphism, or let-polymorphism, which generalizes types to type schemes only at let-bindings, is called Hindley–Milner polymorphism, after J.R. Hindley and Robin Milner, who discovered this idea independently in 1968 and 1977.

The first type inference algorithm for ML, called algorithm W, was presented in 1982 by Luis Damas and Robin Milner [3]. Good introductions to polymorphic type inference include Hancock’s [10] and Schwartzbach’s [13].

The binding level technique for efficient type variable generalization mentioned in Sect. 6.2.2 is due to Didier Rémy [12]. Unification was invented by Alan Robinson [11] in 1965, and is a central implementation technique also in the Prolog language. Type variables are equated efficiently by means of the union-find algorithm [14, Chap. 2], described also in most algorithms textbooks, such as Cormen et al. [2, Chap. 21] or Goodrich and Tamassia [5, Sect. 4.2].

6.8 Exercises

The goals of these exercises are (1) to investigate the interpreter `eval` for the higher-order version of the micro-ML language (in file `HigherFun.fs`), and (2) to understand type ML-style type inference, including the implementation in file `TypeInference.fs`.

Exercise 6.1 Download and unpack `fun1.zip` and `fun2.zip` and build the micro-ML higher-order evaluator as described in file `README.TXT` point E.

Then run the evaluator on the following four programs. Is the result of the third one as expected? Explain the result of the last one:

```
let add x = let f y = x+y in f end
in add 2 5 end
```

```
let add x = let f y = x+y in f end
in let addtwo = add 2
   in addtwo 5 end
end
```

```
let add x = let f y = x+y in f end
in let addtwo = add 2
   in let x = 77 in addtwo 5 end
   end
end
```

```
let add x = let f y = x+y in f end
in add 2 end
```

Exercise 6.2 Add anonymous functions, similar to F#'s `fun x -> ...`, to the micro-ML higher-order functional language abstract syntax:

```
type expr =
  ...
  | Fun of string * expr
  | ...
```

For instance, these two expressions in concrete syntax:

```
fun x -> 2*x
let y = 22 in fun z -> z+y end
```

should parse to these two expressions in abstract syntax:

```
Fun("x", Prim("2", CstI 2, Var "x"))
Let("y", CstI 22, Fun("z", Prim("+", Var "z", Var "y")))
```

Evaluation of a `Fun(...)` should produce a non-recursive closure of the form

```
type value =
  | ...
  | Clos of string * expr * value env    (* (x,body,declEnv) *)
```

In the empty environment the two expressions shown above should evaluate to these two closure values:

```
Clos("x", Prim("2", CstI 2, Var "x"), [])
Clos("z", Prim("+", Var "z", Var "y"), [(y,22)])
```

Extend the evaluator `eval` in file `HigherFun.fs` to interpret such anonymous functions.

Exercise 6.3 Extend the micro-ML lexer and parser specifications in `FunLex.fsl` and `FunPar.fsy` to permit anonymous functions. The concrete syntax may be as in F#: `fun x -> expr` or as in Standard ML: `fn x => expr`, where `x` is a variable. The micro-ML examples from Exercise 6.1 can now be written in these two alternative ways:

```
let add x = fun y -> x+y
in add 2 5 end

let add = fun x -> fun y -> x+y
in add 2 5 end
```

Exercise 6.4 This exercise concerns type rules for ML-polymorphism, as shown in Fig. 6.1.

(i) Build a type rule tree for this micro-ML program (in the `let-body`, the type of `f` should be polymorphic – why?):

```
let f x = 1
in f f end
```